Byte Energy Descent (BED): A Discrete State→Text Decoder for Physics-Based Cognitive Architectures

Irfan Mahir — Independent Researcher, Dhaka, Bangladesh — irfan@furylogic.com

Contents

Byte Energy Descent (BED): A Discrete State→Text Decoder for Physics-Based Cognitive Architectures **1**
Abstract . . . 1
1. Introduction . . . 2
1.1 The state→text failure of Hamiltonian rollout . . . 2
1.2 Why continuous rollout + rounding is decoupled from Φ . . . 2
1.3 Contribution . . . 2
2. Related Work . . . 2
3. Mathematical Formulation . . . 3
3.1 The DCHL potential in cached form . . . 3
3.2 Rank-1 update under a single-byte change . . . 3
3.3 The BED update rule . . . 4
3.4 Temperature-controlled variant . . . 4
3.5 Monotone descent (zero temperature) . . . 4
4. Complexity . . . 4
5. Algorithm . . . 5
6. Comparison to Alternatives . . . 6
7. Empirical Observations (Honest Reading) . . . 6
7.1 BED does what BED should . . . 6
7.2 The outputs are still not English . . . 7
7.3 The decoder is correct; the potential was trained as a classifier . . . 7
8. What is Actually Needed Next: Manifold Sculpting . . . 7
8.1 Negatives as perturbations, not as a class . . . 8
8.2 Why this stops being a classifier . . . 8
8.3 Interaction with BED . . . 8
9. Limitations . . . 8
10. Conclusion . . . 9
References . . . 9

Byte Energy Descent (BED): A Discrete State→Text Decoder for Physics-Based Cognitive Architectures

Irfan Mahir *Independent Researcher, Dhaka, Bangladesh* irfan@furylogic.com

> *"The state is the answer. But only if the potential is sculpted, not classified, and the decoder never leaves the state."*

Abstract

We present **Byte Energy Descent (BED)**, a novel state→text decoder for the DCHL / P-CMA / HCMA family of physics-based cognitive architectures. Prior work in this lineage evolves a continuous cognitive state $\mathbf{c} \in \mathbb{R}^D$ under Hamiltonian dynamics on a learned potential $\Phi(\mathbf{c})$, then reads out an answer by rounding each answer-region float to the nearest byte. We show — analytically and empirically — that this pointwise rounding is decoupled from Φ: the continuous minimiser of Φ has no obligation to land on the integer byte lattice, and the resulting bytes drift toward the marginal byte mean of the training corpus (an unconditional low-order statistic) instead of a prompt-conditioned coherent completion.

BED replaces both the continuous roll-out and the pointwise rounding with a single **discrete coordinate descent on the byte lattice** driven by Φ itself. The state stays on $\{0, 1, \dots, 255\}^D$ at every intermediate configuration, so it always decodes to real text. Each answer position i is updated by evaluating Φ at all 256 candidate byte values and choosing the argmin, with positions swept in a randomised order (i.e. **not** left-to-right, so BED is not autoregressive and not sequential). A rank-1 cache of pre-activations makes each 256-way candidate evaluation cost $O(H)$ instead of $O(H \cdot D)$, so a full sweep costs $O(D_{\text{answer}} \cdot H)$ and the whole decoder runs in a few sweeps.

We derive the update rule, prove monotone non-increase of Φ at zero temperature, and give the exact implementation in `src/hcma.hpp:think_bytes`. We report honest empirical results on a small model trained on the Pile: BED does produce prompt-dependent answers (each prompt gives a distinct Φ_{final} and distinct byte distribution), but its outputs are not English at this training scale. We locate the residual failure precisely — the contrastive learning signal used to shape Φ is too weak once trivial negatives are exhausted — and propose **hard-negative mining from the corpus itself** (wrong text completions of the same prompt) as the next necessary step for the architecture to develop the coherence basins BED needs.

---

1. Introduction

1.1 The state→text failure of Hamiltonian rollout

The HCMA/P-CMA/DCHL architecture stores its output as a cognitive state $\mathbf{c} \in [0, 1]^D$ obtained by mapping each byte b to $b/256$. Inference splits $\mathbf{c}$ into a *prompt region* (frozen) and an *answer region* (evolved). Hamiltonian dynamics on the learned potential $\Phi(\mathbf{c})$ then push the answer region toward a low-energy configuration:

$$\dot{c}_i = p_i, \qquad \dot{p}_i = -\partial_i \Phi(\mathbf{c}), \qquad i \in \text{answer region}.$$

The classical decoder reads out an answer via

$$\text{byte}_i = \text{round}(c_i \cdot 256).$$

We observe that this pipeline reliably fails to produce meaningful text — even when Φ has been trained to strongly separate coherent from incoherent bytes. The output collapses to a repetitive stream of bytes near the corpus mean (around ASCII 98–113, `b`–`q`), regardless of the prompt.

1.2 Why continuous rollout + rounding is decoupled from Φ

Two independent problems compose:

Problem A: the gradient in fp32 space has no reason to point at integer bytes. Φ is trained on real byte-valued inputs (pixels of the byte lattice), but *evaluated* at every point in $[0,1]^D$. The continuous minimiser of Φ under the smooth extension is not, in general, one of the $256^{D_{\text{answer}}}$ integer configurations. Hamiltonian descent finds the smooth minimum, which is typically a byte-mean vector.

Problem B: pointwise rounding erases prompt conditioning. Even if descent found a good continuous point, `round(c_i · 256)` treats each c_i independently. Any correlations between adjacent bytes — which is where coherence lives — are lost in the last step.

The result is that the answer is drawn from the *marginal* byte distribution of the training corpus, not the *conditional* distribution given the prompt.

1.3 Contribution

We propose **Byte Energy Descent (BED)**. BED never leaves the byte lattice. It updates each answer byte to the argmin of Φ over the 256 candidate values, given the current values of all other bytes (prompt + rest of answer). Positions are updated in a randomised order across each sweep, so the update rule is neither autoregressive nor sequential.

BED closes both problems at once: 1. Every intermediate state, and the final state, is composed of real bytes. There is no rounding step to disrupt Φ-conditioned coupling. 2. Each byte's choice sees the whole state through Φ, so joint / conditional structure is preserved.

Complexity is kept tractable by a rank-1 cache: the shift in Φ from changing byte i from v_{old} to v is a scalar function of $(v - v_{\text{old}})$ and a single column of W_1.

---

2. Related Work

BED sits at the intersection of three literatures.

Autoregressive decoding (Transformer LMs, causal RNNs). Text is generated by sampling $y_t \sim P(\cdot \mid y_{<t}, x)$ token-by-token, left-to-right, over a learned vocabulary. BED does *not* condition on a causal prefix and does *not* have a vocabulary — the 256 candidates at each position are the raw byte values, and positions are visited in random order.

Diffusion / score-based decoding. Generate by denoising a continuous latent under $-\nabla \log p$. BED has no noise schedule, no reverse process, and works on the discrete byte lattice rather than a continuous latent.

Iterated Conditional Modes (ICM) [Besag, 1986]. Classical MRF inference: fix all variables but one, set that one to its conditional argmin. BED is exactly ICM applied to the byte-level Markov random field whose energy is the DCHL potential Φ. What is novel here is (i) using a scalar potential learned by decoupled contrastive Hebbian updates as the energy, (ii) the rank-1 caching that keeps per-position cost independent of D, and (iii) the pairing with a prompt-frozen state so BED becomes a conditional text generator.

Greedy energy decoding for byte-level models. Prior byte-level generative work (byte-level LMs, ByT5) still uses autoregressive softmax over 256 output classes. BED replaces the entire softmax head with direct Φ-scoring over candidates and dispenses with the causal mask.

---

3. Mathematical Formulation

3.1 The DCHL potential in cached form

Recall the DCHL potential

$$\Phi(\mathbf{c}) = b_2 + \sum_{j=1}^{H} w_{2,j} \tanh\left(\frac{W_{1,j} \cdot \mathbf{c} + b_{1,j}}{\|[\mathbf{c}, 1]\|}\right).$$

Introduce the pre-activation cache and squared augmented-norm cache

$$\pi_j \;=\; W_{1,j} \cdot \mathbf{c} + b_{1,j}, \qquad n^2 \;=\; 1 + \|\mathbf{c}\|^2.$$

Then

$$\Phi(\mathbf{c}) \;=\; b_2 + \sum_{j} w_{2,j} \tanh\left(\frac{\pi_j}{\sqrt{n^2}}\right).$$

3.2 Rank-1 update under a single-byte change

Let i be an answer position and let v be a candidate byte value ($v = b/256$ for $b \in \{0, \dots, 255\}$). Denote the current value at position i by $c_i = v_{\text{old}}$, and let $\Delta v = v - v_{\text{old}}$.

Changing only c_i perturbs the caches by a single column of W_1 and a single scalar:

$$\pi_j^{(v)} \;=\; \pi_j + W_1[j, i]\, \Delta v, \qquad (n^2)^{(v)} \;=\; n^2 + v^2 - v_{\text{old}}^2.$$

Both quantities are computed in $O(1)$ per hidden unit j. The candidate energy is then

$$\Phi^{(v)}(i) \;=\; b_2 + \sum_{j} w_{2,j} \tanh\left(\frac{\pi_j^{(v)}}{\sqrt{(n^2)^{(v)}}}\right),$$

with total cost $O(H)$ per candidate value.

3.3 The BED update rule

At each answer position i visited in a random order,

$$c_i^* \;=\; \operatorname*{arg\,min}_{v \in \{0,1,\dots,255\}/256} \Phi^{(v)}(i).$$

If the update was accepted ($c_i^* \neq v_{\text{old}}$), the caches are patched in $O(H)$:

$$\pi_j \;\leftarrow\; \pi_j + W_1[j, i]\,(c_i^* - v_{\text{old}}), \qquad n^2 \;\leftarrow\; n^2 + (c_i^*)^2 - v_{\text{old}}^2.$$

3.4 Temperature-controlled variant

For diverse or exploratory generation we replace the argmin by a temperature-controlled softmax over the 256 candidate energies

$$p(v) \;\propto\; \exp\left(-\frac{\Phi^{(v)}(i) - \min_{v'} \Phi^{(v')}(i)}{T}\right).$$

At $T \to 0$ this reduces to the deterministic argmin; at $T \to \infty$ it becomes uniform sampling. Sampling from the *conditional* Boltzmann distribution at each position is a component of a Gibbs sampler for the joint Boltzmann distribution $\propto \exp(-\Phi/T)$ over the byte lattice.

3.5 Monotone descent (zero temperature)

Proposition. At $T = 0$ the sequence $\Phi(\mathbf{c}^{(t)})$ is non-increasing across BED updates.

Proof. Each update replaces c_i by the value that minimises Φ conditional on all other bytes; therefore $\Phi(\mathbf{c}^{(t+1)}) \leq \Phi(\mathbf{c}^{(t)})$. □

BED terminates when a full sweep produces no improvement above a tolerance ϵ, or after a maximum number of sweeps.

---

4. Complexity

Let H be the number of hidden units, $D = D_{\text{prompt}} + D_{\text{answer}}$ the state dimension, and S the number of sweeps.

Step	Cost	Notes
Cache initialisation (π_j, n^2)	$O(HD)$	Once per query
One candidate evaluation	$O(H)$	Uses cached π_j, n^2
One position update (256 candidates + patch)	$O(256H) = O(H)$	Constant factor of 256
One sweep over D_{answer} positions	$O(D_{\text{answer}} \cdot H)$	No D factor
Full BED run (S sweeps)	$O(HD + S \cdot D_{\text{answer}} \cdot H)$	

For a 1B-parameter DCHL model with $H \approx 10^6$, $D_{\text{answer}} = 512$, and $S = 6$ sweeps, BED is $\approx 3 \times 10^9$ operations per query — a few seconds on CPU, sub-second on the RTX 4060 with a straightforward GPU port.

By contrast, autoregressive Transformer decoding of the same D_{answer} tokens with a comparable-width model costs $O(D_{\text{answer}}^2 \cdot d_{\text{model}})$ attention plus $O(D_{\text{answer}} \cdot P)$ MLP work per generated token. BED avoids both the quadratic attention and the KV cache entirely.

---

5. Algorithm

```
Input:  Potential Phi (W1, b1, w2, b2, D_prompt, D_answer, H)
        Prompt string q
        Number of sweeps S, temperature T
Output: Answer string a, phi_initial, phi_final

# 1. Assemble byte-lattice initial state.
c = [byte/256 for byte in bytes(q) padded to D_prompt] ++ [128/256] * D_answer

# 2. Prime the caches.
n2  = 1 + sum(c_i^2 for c_i in c)
for j in 1..H:
    pi_j = W1[j] . c + b1[j]
phi_initial = b2 + sum(w2_j * tanh(pi_j / sqrt(n2)) for j)
```

```
# 3. Coordinate descent on the answer region.
positions = list(range(D_prompt, D_prompt + D_answer))
for sweep in 1..S:
    shuffle(positions)
    for i in positions:
        v_old = c[i]
        for b in 0..255:
            v = b / 256
            dv = v - v_old
            n2_v  = n2 + v*v - v_old*v_old
            inv_n = 1 / sqrt(n2_v)
            phi_v = b2 + sum(w2_j * tanh((pi_j + W1[j,i] * dv) * inv_n) for j)
            phi_cand[b] = phi_v
        if T > 0:
            b_star = softmax_sample(-(phi_cand - min(phi_cand)) / T)
        else:
            b_star = argmin(phi_cand)
        v_new = b_star / 256
        if v_new != v_old:
            # rank-1 cache patch
            for j in 1..H:
                pi_j += W1[j,i] * (v_new - v_old)
            n2 += v_new*v_new - v_old*v_old
            c[i] = v_new
    phi_now = b2 + sum(w2_j * tanh(pi_j / sqrt(n2)) for j)
    if T == 0 and abs(phi_now - phi_prev) < eps:
        break
    phi_prev = phi_now

# 4. Emit the answer region as bytes.
a = bytes(round(c_i * 256) for i in D_prompt..D_prompt+D_answer-1)
return a, phi_initial, phi_now
```

Reference implementation: `src/hcma.hpp` function `think_bytes`.

---

6. Comparison to Alternatives

Property	Autoregressive LM	Diffusion LM	Hamiltonian + round	**BED (ours)**
Output symbol set	learned tokens	pixels / bytes	bytes (post-round)	**bytes (native)**
Left-to-right?	yes	no	no	**no**
Causal mask?	yes	no	no	**no**
Sees whole context per emission?	prefix only	yes	yes	**yes**
Uses a softmax vocabulary head?	yes	no	no	**no**
Continuous relaxation?	no	yes	yes	**no**

Property	Autoregressive LM	Diffusion LM	Hamiltonian + round	**BED (ours)**
Coupled to trained energy?	via cross-entropy	via score	**no** (rounding decouples)	**yes** (direct argmin)
Memory during decoding	$O(N)$ KV	$O(D)$	$O(D)$	$O(H + D)$ (caches)

BED is the only column where every intermediate output is on the native symbol set, no vocabulary is defined, and the decoding step is a direct search on the *same* energy landscape used to train the model.

---

7. Empirical Observations (Honest Reading)

We integrated BED into the reference DCHL implementation (`chatbot.cpp`) and trained potentials of two sizes on the Pile sample from `~/Desktop/datasets`:

- `small.neuron` — $D_{\text{prompt}} = D_{\text{answer}} = 256$, $H = 1024$, 3 epochs.
- `chatbot_v2.neuron` — $D_{\text{prompt}} = D_{\text{answer}} = 256$, $H = 8192$, 30 epochs, hard negatives only (shuffle + splice, no random).

7.1 BED does what BED should

- Every intermediate state decodes to real bytes; there is no rounding step.
- Φ_{final} is deterministically non-increasing at $T = 0$, as the proposition predicts.
- Distinct prompts produce distinct Φ_{final} and distinct byte distributions:

Prompt	Hamiltonian rollout (legacy)	BED (this work)
"The Roman Empire was"	`bbbbb…` (byte 98 flat)	mixed letters/punctuation around ASCII 118–126, distinct per prompt
"In the beginning"	`bbbbb…`	space-heavy pattern reflecting the short prompt
"qwertyuiop1234"	`bbbbb…`	OOD-flagged, byte pattern shifts noticeably

The classical Hamiltonian decoder collapses every prompt to the same saturation byte. BED produces prompt-conditioned outputs whose byte statistics differ.

7.2 The outputs are still not English

This is where we have to be very careful and honest. Every BED output at this training scale is a mixture of adjacent-in-ASCII letters and punctuation, with the right *marginal* byte distribution but no legible words.

The training curve tells us why. Over 30 epochs of `chatbot_v2`:

Symptom	Meaning
$\Delta\Phi \approx 0.1$–0.2 and not growing	Φ is barely distinguishing coherent from incoherent.

Symptom	Meaning
$\|h\| \approx 0.52$ stable	Hidden units are alive but not specialising.
Φ(coherent) wanders without a clear downward trend	The contrastive signal is not providing a consistent gradient direction.

The DCHL Hebbian rule can only sharpen features that already differ between $\mathbf{c}^+$ (coherent window) and $\mathbf{c}^-$ (negative). Our negatives are: random bytes, shuffled bytes, and cross-document splices. All three destroy the token- and phrase-level statistics that make text look like text; the model quickly saturates the "text vs. not-text" contrast and then has nothing left to learn from. Once trivial negatives are exhausted, dPhi collapses to noise.

BED faithfully descends whatever Φ has learned. Right now, that is essentially "text has this byte distribution" — so BED lands on the marginal byte distribution, not on a coherent continuation.

7.3 The decoder is correct; the potential was trained as a classifier

BED is not the bottleneck. The bottleneck is that the potential Φ was, up to this point, being trained on a **text-vs-random binary classification objective** dressed up as contrastive Hebbian learning. Once trivial negatives (random bytes) are exhausted, the training curve collapses to $\Delta\Phi \approx 0.1$ — because there is nothing left for a *classifier* to learn. Even the "harder" variants (shuffle, cross-document splice) are still binary classification: they just move the decision boundary.

A classifier that BED descends into just produces a byte-average of the winning class. Different prompts do get different Φ_{final} because Φ has some prompt-dependent capacity, but the answer bytes come from the class mean, not from anywhere the prompt actually points to. That is precisely pattern matching, and it is the correct behaviour of a well-trained *classifier* under BED — which confirms that BED is doing its job on the input it is given.

---

8. What is Actually Needed Next: Manifold Sculpting

To make BED produce text, Φ must stop being trained as a classifier and start being trained as an **energy landscape whose basins sit on the data manifold**. This is a change in objective, not in machinery: the DCHL Hebbian update itself is unchanged (per-unit contrastive on the hidden layer, pure differential on w_2, ZeroGrad throughout). What changes is what plays the role of $\mathbf{c}^-$.

8.1 Negatives as perturbations, not as a class

For each positive $\mathbf{c}^+$ drawn from the corpus we generate the negative as a *perturbation* of $\mathbf{c}^+$ itself:

$$\mathbf{c}^- \;=\; \mathbf{c}^+ + \eta, \qquad \eta \sim \mathcal{P}(\|\eta\|)$$

The DCHL step then raises $\Phi(\mathbf{c}^-)$ by an amount that, over the corpus, aligns with $\|\eta\|$. There is no "class of negatives" — $\mathcal{P}$ is a *continuous* perturbation distribution, and Φ acquires a basin at $\mathbf{c}^+$ whose *shape* is determined by that distribution. This is the same idea underneath score matching and denoising diffusion, ported to DCHL: instead of learning the density of the data directly, we learn to make its gradient point back to the data.

Two useful perturbation schedules are:

- **Additive noise (score-matching flavour)**: η is small Gaussian noise on the byte-scaled state. Φ is sculpted so $\nabla\Phi(\mathbf{c}^+)$ points *back* to $\mathbf{c}^+$, not toward some class centroid. The DCHL update is exactly the local Hebbian rule from §3, but now $\mathbf{c}^-$ is a smooth perturbation of $\mathbf{c}^+$ rather than a draw from a fixed noise class.
- **Structural coherence perturbation (predictive-coding flavour)**: η replaces the continuation half of $\mathbf{c}^+$ with a valid English continuation drawn from a different document. This is *not* a "wrong class" — it is a specific perturbation that keeps the marginal byte statistics identical and only breaks the joint prompt-continuation structure. DCHL is then forced to allocate hidden units to prompt-continuation *interactions*, which is what BED needs to read out at inference time.

8.2 Why this stops being a classifier

Under this objective, $\mathbf{c}^+$ and $\mathbf{c}^-$ share almost every statistic that would let a classifier separate them; the only signal that survives is the *distance from the data manifold*. Because the perturbation magnitude and direction vary continuously across samples, there is no fixed decision boundary for Φ to converge onto. Instead, Φ acquires a local minimum at every $\mathbf{c}^+$ we show it, with a gradient that points back to that minimum from nearby perturbed states — that is a manifold sculpture, not a decision rule.

At inference, BED lands on a *specific point* on that manifold — a byte-level configuration in the data manifold's neighbourhood of the prompt — instead of the byte mean of a class.

8.3 Interaction with BED

BED itself is unchanged. The two components are cleanly separated: DCHL builds the potential, BED reads it. Moving from a classifier objective to a manifold-sculpting objective is a change in the training loop's negative sampler and in nothing else. When the potential is well-sculpted, the same BED code — every line of it — will start producing text.

---

9. Limitations

1. **Second-order effects between bytes.** BED is coordinate descent — it can get stuck in local minima where any single-byte flip raises Φ but a pair flip would lower it. Extensions: block moves, random restarts, or the temperature-based Gibbs sampler.
2. **Cost of $256 \cdot H$ tanh calls per position.** On CPU, small models (up to a few tens of M parameters) run in seconds; large models need the GPU port.
3. **BED does not create coherence structure.** If Φ has not learned $p(\text{answer} \mid \text{prompt})$, BED cannot invent it. Section 8 is a prerequisite for BED to look like it is "thinking."
4. **UTF-8 and multi-byte characters.** BED treats each byte position independently. UTF-8 continuation bytes are not enforced to be valid multi-byte sequences; corrupted characters are possible mid-stream.

---

10. Conclusion

Byte Energy Descent is the state→text step the DCHL / P-CMA / HCMA lineage has been missing. By keeping the state on the byte lattice at all times and choosing each byte as the direct argmin of the trained potential Φ, BED closes the gap between the physics of the cognitive manifold and the text of the answer. It is not autoregressive, has no vocabulary, has no continuous relaxation, and is coupled to the same Φ that DCHL learns — so any improvement in Φ shows up as an improvement in the decoded text, with no decoder-side changes.

The current implementation cleanly demonstrates two things:

- The decoder itself is correct: prompts produce distinct Φ_{final}, monotone descent holds, and complexity is $O(HD + SD_{\text{answer}}H)$.
- The potential was previously trained as a *binary classifier* (text vs random / shuffle / splice), and BED faithfully descends into that classifier's class centroid — which is pattern matching, not thinking.

The next step is to stop training Φ as a classifier and start training it as a **manifold sculpture**: negatives are continuous perturbations of the positive, not draws from a separate class. The DCHL update rule is unchanged; only the negative sampler changes. When it lands, the same BED code — every line of it — will start producing text.

References

[1] Besag, J. (1986). On the Statistical Analysis of Dirty Pictures. *J. R. Statist. Soc. B.* — Iterated Conditional Modes.

[2] Mahir, I. (2026). Predictive Cognitive Manifold Architecture (P-CMA). *arXiv.*

[3] Mahir, I. (2026). Cognitive Continuum Hypothesis (CCH). *arXiv.*

[4] Mahir, I. (2026). Hierarchical Cognitive Manifold Architecture (HCMA). *arXiv.*

[5] Mahir, I. (2026). ZeroGrad Cognitive Engine. *arXiv.*

[6] Mahir, I. (2026). Arch-Native Quantization (ANQ). *arXiv.*

[7] LeCun, Y. et al. (2006). A Tutorial on Energy-Based Learning.

[8] Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS.*

[9] Hoogeboom, E. et al. (2021). Argmax Flows and Multinomial Diffusion.

[10] Xue, L. et al. (2022). ByT5: Towards a token-free future with pre-trained byte-to-byte models.

BED: the state is the answer, and the decoder never leaves the state.